Relazione di Informatica

Sicurezza Digitale e Crittografia

Analisi teorica e implementazione pratica
del sistema **Nebulae**

Modulo	Mese 1 — Fondamenta della Crittografia
Anno accademico	2025–2026
Disciplina	Sicurezza Informatica

Il documento analizza i concetti teorici della crittografia moderna e ne mostra l'applicazione concreta nel progetto **Nebulae**, un sistema di messaggistica P2P post-quantistico implementato in Rust e Python.

Indice

1 Introduzione alla Sicurezza Informatica e alla Crittografia **3**

1.1 Cos'è la sicurezza informatica 3

1.2 Cos'è la crittografia 3

1.3 La Triade CIA 4

2 Alice, Bob, Eve e Mallory: il Modello di Minaccia Classico **6**

2.1 I personaggi del protocollo 6

2.2 Attacchi passivi: Eve 7

2.3 Attacchi attivi: Mallory 7

3 Crittografia Simmetrica e Asimmetrica **8**

3.1 Crittografia simmetrica 8

3.2 Crittografia asimmetrica 9

4 Breve Storia della Crittografia **11**

4.1 Dalle origini all'antichità 11

4.2 Il Cifrario di Cesare 11

4.3 La Seconda Guerra Mondiale: Lorenz e Colossus 12

4.4 L'era informatica: DES, AES e RSA 12

4.5 I fondamenti matematici della crittografia moderna 13

5 Crittografia Simmetrica Avanzata **14**

5.1 Stream cipher: XOR con keystream 14

5.2 Block cipher: AES 14

5.3 ChaCha20-Poly1305 15

6 Crittografia Asimmetrica Avanzata **16**

6.1 RSA: fattorizzazione come problema difficile 16

6.2 Diffie-Hellman e lo scambio di chiavi 16

6.3 Curve Ellittiche (ECC) e X25519 17

7 Firme Digitali e Funzioni Hash **18**
7.1 Funzioni hash crittografiche 18
7.2 HMAC: autenticazione con hash 18
7.3 Firme digitali: Ed25519 19

8 Il Progetto NEBULAE: Implementazione Pratica **20**
8.1 Architettura generale 20
8.2 Il nucleo crittografico in Rust 20
8.2.1 Generazione dell'identità 21
8.2.2 Incapsulamento ibrido post-quantistico 22
8.2.3 Il Double Ratchet 23
8.2.4 Derivazione della chiave master: PBKDF2 24
8.3 Gestione delle chiavi in Python 25
8.3.1 Il sistema a doppia password 25
8.3.2 Azzeramento sicuro della memoria 25
8.4 Database cifrato 26
8.4.1 Header Blindness: indice HMAC-SHA3 26
8.4.2 Modalità amnesica 27
8.5 Rete P2P su Tor 27
8.5.1 Padding adattivo (chaff traffic) 28
8.5.2 Sequenza di handshake 28
8.6 Panic Wipe: autodistruzione di emergenza 28

9 Conclusioni **30**
9.1 Il filo conduttore della crittografia moderna 30
9.2 NEBULAE come sintesi applicata 30
9.3 Prospettive future 31

Capitolo 1

Introduzione alla Sicurezza Informatica e alla Crittografia

1.1 Cos'è la sicurezza informatica

La **sicurezza informatica** è l'insieme strutturato di tecnologie, processi e pratiche progettate per proteggere sistemi, reti, programmi, dispositivi e dati da attacchi digitali, accessi non autorizzati e minacce informatiche di varia natura. La sua rilevanza è cresciuta in modo esponenziale con la digitalizzazione di ogni aspetto della vita contemporanea: dalle transazioni finanziarie alle comunicazioni personali, dalla gestione sanitaria alle infrastrutture critiche nazionali.

Proteggere correttamente un sistema informatico implica affrontare tre categorie fondamentali di rischio:

1. **Accesso non autorizzato**: soggetti privi di permesso accedono a risorse o informazioni riservate.

2. **Alterazione dei dati**: le informazioni vengono modificate durante la trasmissione o l'archiviazione senza che i legittimi proprietari ne siano consapevoli.

3. **Impersonificazione**: un attaccante si spaccia per un soggetto legittimo al fine di ingannare sistemi o persone.

1.2 Cos'è la crittografia

La **crittografia** è la scienza e la pratica di proteggere le informazioni trasformandole in un formato intelligibile soltanto a chi possiede la chiave di decifratura

appropriata. Etimologicamente il termine deriva dal greco *kryptós* (nascosto) e *gráphein* (scrivere): letteralmente, *scrittura nascosta*.

In senso moderno la crittografia non si limita alla sola cifratura dei messaggi, ma costituisce il fondamento tecnico di numerosi meccanismi di sicurezza: firme digitali, protocolli di autenticazione, scambio sicuro di chiavi e verifica dell'integrità dei dati.

1.3 La Triade CIA

Il paradigma di riferimento della sicurezza informatica moderna è la **Triade CIA** (*Confidentiality, Integrity, Authenticity*). I tre principi sono tra loro complementari e nessuno è sufficiente da solo a garantire una comunicazione sicura.

Tabella 1.1: I tre pilastri della Triade CIA.

Pilastro	Descrizione
Confidenzialità	Solo i soggetti autorizzati possono leggere il contenuto del messaggio. Realizzata mediante cifratura simmetrica (AES, ChaCha20) o asimmetrica (RSA, X25519).
Integrità	I dati non sono stati alterati durante la trasmissione o l'archiviazione. Garantita dalle funzioni di hash crittografico (SHA-256, BLAKE3): qualsiasi modifica, anche di un solo bit, produce un hash completamente diverso (*effetto valanga*).
Autenticità	Il destinatario può verificare l'identità del mittente. Le firme digitali (Ed25519, RSA) associano il messaggio in modo crittograficamente verificabile alla chiave privata del mittente.

Non ripudio. A questi tre principi si aggiunge spesso il *non ripudio*: il mittente non può negare di aver inviato un determinato messaggio, poiché la firma digitale è matematicamente legata alla sua e soltanto alla sua chiave privata.

Fonti:

- https://securityhub.it/cos-e-la-sicurezza-informatica/ [10/03/2026]
- https://www.acn.gov.it/portale/documents/20119/85999/ACN_LG_Introduzione.pdf [10/03/2026]

Tabella 1.2: Panoramica dei tipi principali di crittografia.

Tipo	**Algoritmi tipici**	**Esempio d'uso**
Simmetrica	AES, ChaCha20, DES	Cifratura bulk dei dati
Asimmetrica	RSA, X25519, Kyber	Scambio di chiavi, TLS
Hash (non reversibile)	SHA-256, BLAKE3	Verifica integrità, password
Firma digitale	Ed25519, ECDSA	Autenticazione identità

Capitolo 2

Alice, Bob, Eve e Mallory: il Modello di Minaccia Classico

2.1 I personaggi del protocollo

Nella letteratura crittografica, i protocolli di sicurezza vengono tradizionalmente descritti attraverso un cast fisso di personaggi fittizi, ciascuno dei quali rappresenta un ruolo preciso nella comunicazione o nell'attacco. Questa convenzione, introdotta negli anni Settanta, permette di formalizzare le ipotesi di minaccia in modo intuitivo e universalmente comprensibile.

alice — la mittente Origina il messaggio. Tutti i meccanismi di cifratura, firma e autenticazione prendono avvio dalla sua intenzione di comunicare qualcosa di riservato a Bob.

bob — il destinatario Riceve il messaggio ed è il solo soggetto autorizzato a leggerlo. Deve potersi fidare che ciò che riceve provenga effettivamente da Alice e non sia stato alterato in transito.

eve — l'*eavesdropper* passivo Il nome deriva dall'inglese *eavesdropping* (origliare). Eve osserva il canale di comunicazione senza intervenire: registra e analizza i messaggi intercettati. Minaccia la **confidenzialità**.

mallory — l'attaccante attivo Può modificare i messaggi in transito, iniettarne di falsi, bloccare la consegna o impersonare Alice nei confronti di Bob e viceversa. Minaccia **integrità** e **autenticità**.

2.2 Attacchi passivi: Eve

Un **attacco passivo** è caratterizzato dall'assenza di qualsiasi modifica al flusso di comunicazione. Eve osserva, registra e analizza; non altera. Ciò nonostante, la sua azione può essere devastante: un messaggio non cifrato trasmesso su un canale non protetto è completamente esposto.

Tra le tecniche di attacco passivo più rilevanti si annoverano:

- **Sniffing**: cattura del traffico di rete con strumenti come Wireshark.
- **Traffic analysis**: anche senza decifrare il contenuto, l'analisi di frequenza, dimensione e orari dei messaggi può rivelare informazioni sensibili sul comportamento degli utenti.
- **Replay analysis**: osservazione di pattern ripetuti per dedurre la struttura del protocollo.

Principio fondamentale. Ogni sistema di comunicazione sicuro deve *presumere* la presenza di Eve sul canale. Non è sufficiente sperare che nessuno ascolti: occorre progettare il protocollo in modo che, anche in presenza di un osservatore passivo, il contenuto del messaggio rimanga incomprensibile.

2.3 Attacchi attivi: Mallory

Un **attacco attivo** presuppone la capacità di Mallory di interporsi tra Alice e Bob, modificando, ritardando, replicando o falsificando i messaggi. L'attacco più classico è il **Man-in-the-Middle (MITM)**: Mallory si presenta ad Alice come se fosse Bob, e a Bob come se fosse Alice, intercettando e potenzialmente modificando ogni scambio.

Implicazione progettuale. La sola cifratura non è sufficiente a contrastare Mallory. Se Alice cifra il messaggio con una chiave che crede essere di Bob, ma che in realtà appartiene a Mallory, la cifratura non fornisce alcuna garanzia. È necessaria l'**autenticazione delle chiavi**, realizzata attraverso firme digitali e infrastrutture a chiave pubblica (PKI).

Fonti:

- https://en.wikipedia.org/wiki/Alice_and_Bob [10/03/2026]
- https://www.cybersecurity360.it/cultura-cyber/eve-mallory-trent-i-personaggi [10/03/2026]

Capitolo 3

Crittografia Simmetrica e Asimmetrica

3.1 Crittografia simmetrica

Nella **crittografia simmetrica** la medesima chiave segreta K viene impiegata sia per cifrare sia per decifrare il messaggio. Il termine simmetrica deriva proprio da questa dualità: il processo è simmetrico rispetto alla chiave.

$$C = E_K(M) \qquad M = D_K(C)$$

dove M è il testo in chiaro (*plaintext*), C è il testo cifrato (*ciphertext*), E e D sono le funzioni di cifratura e decifratura.

Vantaggi. La crittografia simmetrica è computazionalmente efficiente e adatta alla cifratura di grandi volumi di dati. Algoritmi moderni come AES-256-GCM e ChaCha20-Poly1305 offrono un elevato livello di sicurezza con latenze nell'ordine dei nanosecondi su hardware contemporaneo.

La robustezza e la lunghezza della chiave. La sicurezza degli algoritmi simmetrici è direttamente proporzionale alla lunghezza della chiave: con 40 bit si ottiene una chiave considerata debole, mentre con 128 bit si raggiunge un livello di sicurezza tale che solo sistemi di crittoanalisi militare avanzata potrebbero tentare di violare in tempi ragionevoli. Lo standard attuale per applicazioni ad alta sicurezza è AES-256 (256 bit).

Il problema della distribuzione delle chiavi. Il punto debole fondamentale della crittografia simmetrica è la necessità di condividere la chiave K tra mittente e

destinatario *prima* di poter comunicare in modo sicuro. Se il canale di distribuzione è insicuro, Eve potrebbe intercettare la chiave e decifrare tutti i messaggi successivi.

Formalizzando: con N utenti che vogliono comunicare in modo *pairwise* sicuro, il numero di chiavi necessarie cresce come:

$$\binom{N}{2} = \frac{N(N-1)}{2}$$

Per un milione di utenti, ciò richiederebbe circa 5×10^{11} chiavi distinte — un problema logistico insostenibile.

3.2 Crittografia asimmetrica

La **crittografia asimmetrica** (o a *chiave pubblica*) risolve il problema della distribuzione introducendo una **coppia di chiavi matematicamente correlate**: una chiave pubblica K_{pub}, liberamente distribuibile, e una chiave privata K_{priv}, mantenuta segreta.

$$C = E(K_{\text{pub}},\, M) \qquad M = D(K_{\text{priv}},\, C)$$

Funzione Trapdoor. Il cuore della crittografia asimmetrica è la *funzione trapdoor*: una funzione f facilmente calcolabile in avanti ($f(x)$ da x), ma computazionalmente intrattabile da invertire (x da $f(x)$) senza una informazione segreta aggiuntiva (la *trapdoor*). Esempi classici: fattorizzazione di interi grandi (RSA), problema del logaritmo discreto (Diffie-Hellman, ECC).

Svantaggi e soluzione ibrida. Gli algoritmi asimmetrici sono computazionalmente onerosi rispetto ai simmetrici (da 10 a 1000 volte più lenti). In pratica, i protocolli moderni adottano uno schema **ibrido**: la crittografia asimmetrica viene utilizzata esclusivamente per negoziare in modo sicuro una *session key* simmetrica, con cui viene poi cifrata la comunicazione effettiva. Questa è esattamente la strategia adottata da TLS, Signal e **Nebulae**.

Fonte:

- https://it.wikipedia.org/wiki/Differenza_tra_cifratura_simmetrica_e_asimmetrica [11/03/2026]

Tabella 3.1: Confronto tra crittografia simmetrica e asimmetrica.

Caratteristica	**Simmetrica**	**Asimmetrica**
Chiavi	Una sola chiave condivisa	Coppia pubblica/privata
Velocità	Molto alta	Lenta (10–1000×)
Distribuzione	Problematica	Risolta (chiave pubblica)
Autenticazione	Non nativa	Nativa (firme digitali)
Uso tipico	Cifratura bulk dei dati	Scambio chiavi, firme
Esempi	AES-256, ChaCha20	RSA, X25519, Kyber

Capitolo 4

Breve Storia della Crittografia

4.1 Dalle origini all'antichità

La crittografia ha origini antichissime, attestate già nelle civiltà egizia e greca. Le prime tecniche erano fondamentalmente di due tipi:

Cifrari a trasposizione Le lettere del messaggio vengono riorganizzate secondo uno schema predefinito. Esempio: `hello world` $\rightarrow$ `ehlol owrdl`.

Cifrari a sostituzione Ogni lettera viene sostituita con un'altra secondo una regola fissa. Esempio: `fly at once` $\rightarrow$ `gmz bu podf` (sostituzione con la lettera successiva nell'alfabeto).

Entrambe le categorie, nelle loro forme più semplici, offrono scarsa sicurezza: la **analisi delle frequenze** — introdotta dal matematico arabo Al-Kindi nel IX secolo — permette di rompere la maggior parte dei cifrari a sostituzione monoalfabetica in pochi minuti.

4.2 Il Cifrario di Cesare

Il primo cifrario di sostituzione documentato con precisione storica è il **cifrario di Cesare**, attribuito a Giulio Cesare (I sec. a.C.). Ogni lettera del testo in chiaro viene sostituita con quella che si trova tre posizioni più avanti nell'alfabeto:

$$E(x) = (x + 3) \bmod 26 \qquad D(y) = (y - 3) \bmod 26$$

Nonostante la sua semplicità, questo cifrario è storicamente significativo poiché introduce il concetto di **chiave** (lo spostamento, pari a 3) come parametro separato dall'algoritmo.

4.3 La Seconda Guerra Mondiale: Lorenz e Colossus

Con l'avvento della meccanizzazione, la complessità crittografica aumentò considerevolmente. La Germania nazista impiegò la macchina **Lorenz** per le comunicazioni strategiche di massimo livello tra il comando tedesco e i fronti militari, rendendo estremamente difficile agli Alleati intercettare e decifrare i messaggi senza conoscerne la configurazione.

In risposta, a **Bletchley Park** fu sviluppato **Colossus**: il primo computer digitale e programmabile completamente elettronico al mondo, progettato specificamente per la crittoanalisi dei cifrari Lorenz. Colossus rappresenta un punto di discontinuità fondamentale: per la prima volta, la crittoanalisi divenne un'operazione computazionale automatizzata, aprendo la strada alla crittografia moderna basata su elaborazione elettronica.

4.4 L'era informatica: DES, AES e RSA

Con la diffusione dei calcolatori elettronici nel secondo dopoguerra, la crittografia subì una trasformazione radicale. Gli algoritmi non erano più eseguiti a mano o da macchine elettromeccaniche, ma da software su hardware universali.

Tabella 4.1: Evoluzione degli standard crittografici moderni.

Anno	**Standard**	**Significato**
1977	DES	Primo standard crittografico pubblico (56-bit, oggi obsoleto)
1977	RSA	Primo algoritmo asimmetrico pratico; basato sulla fattorizzazione
1991	PGP	Prima applicazione diffusa di crittografia asimmetrica per email
2001	AES	Sostituto del DES; standard simmetrico dominante (128/256-bit)
2016	X25519/Ed25519	Curve ellittiche ad alta efficienza; adottati da Signal, TLS 1.3
2024	ML-KEM-768	Standard NIST post-quantistico (Kyber); resistente ai computer quantistici

4.5 I fondamenti matematici della crittografia moderna

Alcune tecniche crittografiche moderne sono sicure nella misura in cui certi **problemi matematici** rimangono computazionalmente intrattabili. I principali fondamenti matematici sono:

- **Fattorizzazione di interi** (RSA): dato il prodotto $n = p \cdot q$ di due grandi numeri primi, è computazionalmente difficile ricavare p e q.
- **Logaritmo discreto** (Diffie-Hellman, DSA): dato $g^x \bmod p$, è computazionalmente difficile ricavare x.
- **Problema ECDLP** (X25519, Ed25519): versione del logaritmo discreto su curve ellittiche; offre sicurezza equivalente a RSA con chiavi molto più corte.
- **Learning With Errors — LWE** (Kyber/ML-KEM): fondamento della crittografia post-quantistica; ritenuto resistente anche ai computer quantistici.

Nota sulla dimostrabilità. Nessuno dei sistemi crittografici citati è dimostrabilmente sicuro in senso assoluto. L'unica eccezione teorica è il *cifrario di Vernam* (one-time pad), la cui sicurezza incondizionata è stata dimostrata da Claude Shannon nel 1949. Per tutti gli altri sistemi, la sicurezza si fonda sull'attuale assenza di algoritmi efficienti per risolvere i problemi matematici sottostanti.

Fonte:

- https://en.wikipedia.org/wiki/Cryptography#History [11/03/2026]

Capitolo 5

Crittografia Simmetrica Avanzata

5.1 Stream cipher: XOR con keystream

Uno **stream cipher** cifra il testo in chiaro un byte alla volta, applicando l'operazione XOR con un flusso pseudocasuale di bit detto *keystream*, generato deterministicamente a partire dalla chiave segreta e da un nonce:

$$C_i = M_i \oplus K_i \qquad M_i = C_i \oplus K_i$$

La proprietà fondamentale dell'XOR è che se il keystream è indistinguibile da un flusso veramente casuale, il testo cifrato non rivela alcuna informazione sul testo in chiaro. La sicurezza dipende interamente dalla qualità del generatore di keystream.

5.2 Block cipher: AES

Un **block cipher** divide il messaggio in blocchi di dimensione fissa (128 bit per AES) e li cifra applicando iterativamente una serie di trasformazioni matematiche (*round*). AES con chiave a 256 bit esegue 14 round, ciascuno composto da quattro operazioni: *SubBytes*, *ShiftRows*, *MixColumns*, *AddRoundKey*.

Modalità operative. La modalità con cui AES viene applicato a messaggi più lunghi di un blocco è critica per la sicurezza:

- **ECB (Electronic Codebook)**: ogni blocco è cifrato indipendentemente. *Non sicura*: blocchi identici producono cifrature identiche, rivelando pattern nel plaintext.

- **CBC (Cipher Block Chaining)**: ogni blocco viene combinato via XOR con il precedente prima della cifratura. Richiede un IV casuale; vulnerabile agli attacchi padding oracle se non combinata con un MAC.
- **GCM (Galois/Counter Mode)**: modalità autenticata (AEAD). Combina cifratura e autenticazione in un unico passo. Standard raccomandato per la maggior parte delle applicazioni.

5.3 ChaCha20-Poly1305

ChaCha20-Poly1305 è la principale alternativa ad AES-GCM, progettata da Daniel J. Bernstein. È uno stream cipher (ChaCha20) combinato con un MAC (Poly1305) per ottenere autenticazione. I vantaggi rispetto ad AES-GCM sono:

- **Velocità su hardware senza AES-NI**: su CPU prive dell'istruzione hardware AES, ChaCha20 è significativamente più rapido.
- **Resistenza ai timing attack**: l'implementazione è naturalmente *constant-time*, senza look-up table.
- **Semplicità implementativa**: riduce il rischio di errori nell'implementazione.

Nebulae utilizza ChaCha20-Poly1305 per tutta la cifratura dei messaggi, sia a livello di trasporto (Double Ratchet) sia per lo storage su database.

Capitolo 6

Crittografia Asimmetrica Avanzata

6.1 RSA: fattorizzazione come problema difficile

RSA (Rivest, Shamir, Adleman, 1977) è il primo algoritmo asimmetrico pratico. La generazione delle chiavi procede come segue:

1. Scegliere due grandi numeri primi p e q.
2. Calcolare $n = p \cdot q$ e $\phi(n) = (p-1)(q-1)$.
3. Scegliere e coprimo con $\phi(n)$; calcolare $d = e^{-1} \bmod \phi(n)$.
4. Chiave pubblica: (n, e). Chiave privata: (n, d).

$$\text{Cifratura:} \quad C = M^e \bmod n \qquad \text{Decifratura:} \quad M = C^d \bmod n$$

La sicurezza di RSA si fonda sulla difficoltà computazionale di fattorizzare n in p e q. Per chiavi a 2048 bit, il migliore algoritmo classico noto richiederebbe un tempo astronomico. Tuttavia, l'algoritmo di Shor, eseguibile su un computer quantistico sufficientemente potente, risolve la fattorizzazione in tempo polinomiale, rendendo RSA vulnerabile nell'era post-quantistica.

6.2 Diffie-Hellman e lo scambio di chiavi

Il **protocollo Diffie-Hellman** (1976) risolve elegantemente il problema dello scambio di chiavi su un canale pubblico. Il meccanismo si basa sul problema del logaritmo discreto:

1. Alice e Bob concordano pubblicamente un gruppo ciclico di ordine primo p e un generatore g.
2. Alice sceglie un segreto a e invia $A = g^a \bmod p$.
3. Bob sceglie un segreto b e invia $B = g^b \bmod p$.
4. Entrambi calcolano il segreto condiviso: $S = B^a \bmod p = A^b \bmod p = g^{ab} \bmod p$.

Eve, pur osservando A e B, non riesce a ricavare S senza risolvere il logaritmo discreto (a da $g^a \bmod p$), che è computazionalmente intrattabile per p sufficientemente grande.

6.3 Curve Ellittiche (ECC) e X25519

Le **curve ellittiche** permettono di ottenere la stessa sicurezza di RSA e DH classico con chiavi molto più corte. Una curva ellittica su un campo finito è definita dall'equazione:

$$y^2 = x^3 + ax + b \pmod p$$

Su questi gruppi si definisce un'operazione di moltiplicazione scalare: dato un punto base G e uno scalare k, calcolare $Q = k \cdot G$ è facile, ma ricavare k da Q e G (il problema ECDLP) è computazionalmente intrattabile.

X25519 (Curve25519) è la curva ellittica progettata da Bernstein per essere sicura, efficiente e immune da timing attack. È usata in TLS 1.3, Signal, WhatsApp e — come vedremo — in **Nebulae**.

Capitolo 7

Firme Digitali e Funzioni Hash

7.1 Funzioni hash crittografiche

Una **funzione hash crittografica** H mappa un input di lunghezza arbitraria in un output di lunghezza fissa (*digest*), con le seguenti proprietà:

1. **One-way (preimage resistance)**: dato h, è computazionalmente impossibile trovare m tale che $H(m) = h$.
2. **Collision resistance**: è computazionalmente impossibile trovare due messaggi distinti $m_1 \neq m_2$ con $H(m_1) = H(m_2)$.
3. **Avalanche effect**: una minima variazione dell'input (anche un solo bit) produce un output completamente diverso.

Gli algoritmi più importanti sono SHA-256 (output 256 bit, standard NIST), SHA-3 (basato su Keccak, alternativa strutturalmente diversa da SHA-2) e BLAKE3 (moderno, molto veloce, usato da **Nebulae** per combinare segreti condivisi nella fase di handshake).

7.2 HMAC: autenticazione con hash

HMAC (*Hash-based Message Authentication Code*) combina una funzione hash con una chiave segreta per produrre un codice di autenticazione:

$$\mathrm{HMAC}(K, M) = H\Big((K \oplus \mathrm{opad}) \,\|\, H((K \oplus \mathrm{ipad}) \,\|\, M)\Big)$$

Permette al destinatario di verificare sia l'integrità del messaggio sia che esso provenga da qualcuno che conosce la chiave K. **Nebulae** utilizza HMAC-SHA3-

256 come indice cieco nel database per implementare l'Header Blindness (si veda la Sezione 8.4.1).

7.3 Firme digitali: Ed25519

Una **firma digitale** permette ad Alice di associare a un messaggio una prova crittograficamente verificabile della sua provenienza:

1. Alice calcola $\sigma = \mathrm{Sign}(K_{\mathrm{priv}}, M)$.
2. Bob verifica: $\mathrm{Verify}(K_{\mathrm{pub}}, M, \sigma) = \texttt{true}$.

Ed25519 è lo schema di firma basato su Curve25519 (variante Edwards). Produce firme a 64 byte, è molto veloce e offre implementazioni intrinsecamente sicure contro timing attack. **Nebulae** usa Ed25519 per l'autenticazione dell'identità di ogni nodo.

Capitolo 8

Il Progetto NEBULAE: Implementazione Pratica

8.1 Architettura generale

Nebulae è un sistema di messaggistica **peer-to-peer** progettato per massimizzare la sicurezza della comunicazione tra due utenti. A differenza dei sistemi centralizzati (WhatsApp, Telegram), non esiste alcun server intermedio che possa essere compromesso, intercettato o costretto a rivelare i dati degli utenti.

Le caratteristiche architetturali fondamentali sono:

- **Rete Tor**: tutto il traffico transita su Tor, garantendo anonimato e resistenza alla sorveglianza a livello di rete.
- **Crittografia ibrida post-quantistica**: ogni sessione viene inizializzata con X25519 + ML-KEM-768 (Kyber), resistente ai computer quantistici.
- **Double Ratchet**: ogni messaggio ha una chiave di cifratura unica; lo stesso algoritmo usato da Signal.
- **Nucleo Rust**: le primitive crittografiche sono scritte in Rust, che garantisce memory safety senza garbage collector, eliminando intere classi di vulnerabilità.

8.2 Il nucleo crittografico in Rust

Il modulo Rust espone le primitive crittografiche a Python tramite la libreria **PyO3**, che compila codice Rust come modulo nativo Python. Questo approccio

Tabella 8.1: Architettura a strati di **Nebulae**.

Livello	**File**	**Responsabilità**
Interfaccia	`main.py`, `gui.py`	CLI Rich / GUI grafica
Applicazione	`app.py`	Orchestrazione subsistemi, canary, panic
Rete	`network.py`	P2P su Tor, handshake, chaff traffic
Storage	`database.py`	SQLCipher, indice HMAC, modalità amnesica
Crypto	`crypto_manager.py`	Sessioni, doppia password, zeroizzazione
Nucleo Rust	`lib.rs`	ChaCha20, X25519, Ed25519, Kyber, Ratchet

combina la flessibilità di Python per la logica applicativa con le garanzie di sicurezza della memoria offerte da Rust per le operazioni crittografiche.

8.2.1 Generazione dell'identità

Ogni nodo **Nebulae** possiede un'identità composta da due coppie di chiavi: X25519 per lo scambio Diffie-Hellman (cifratura) e Ed25519 per le firme digitali (autenticazione).

Listing 8.1: Generazione dell'identità in Rust (`lib.rs`).

```
#[pyfunction]
fn generate_identity(py: Python<'_>) -> PyResult<PyObject> {
    // Coppia Ed25519 per firme digitali
    let signing_key   = SigningKey::generate(&mut OsRng);
    let verifying_key = signing_key.verifying_key();

    // Coppia X25519 per scambio Diffie-Hellman
    let static_secret = StaticSecret::random_from_rng(OsRng);
    let dh_public     = PublicKey::from(&static_secret);

    let dict = pyo3::types::PyDict::new(py);
    dict.set_item("ed25519_private",
        PyBytes::new(py, signing_key.to_bytes().as_ref()))?;
    dict.set_item("ed25519_public",
        PyBytes::new(py, verifying_key.to_bytes().as_ref()))?;
    dict.set_item("x25519_private",
        PyBytes::new(py, static_secret.to_bytes().as_ref()))?;
    dict.set_item("x25519_public",
```

```
        PyBytes::new(py, dh_public.as_bytes().as_ref()))?;
    Ok(dict.into())
}
```

8.2.2 Incapsulamento ibrido post-quantistico

Il punto crittograficamente più innovativo di **Nebulae** è l'impiego di uno **schema ibrido** X25519 + ML-KEM-768 (Kyber). La ragione è la seguente: X25519 è sicuro contro gli attacchi classici, ma vulnerabile all'algoritmo di Shor su computer quantistico. Kyber, basato sul problema LWE, è resistente anche ai computer quantistici. Lo schema ibrido garantisce sicurezza in entrambi gli scenari, seguendo la strategia raccomandata dal NIST nella transizione post-quantistica.

Listing 8.2: Incapsulamento ibrido X25519 + Kyber-768 (`lib.rs`).

```
#[pyfunction]
fn hybrid_encapsulate(
    py: Python<'_>,
    x25519_peer_pub: &[u8],
    kyber_peer_pub:  &[u8],
) -> PyResult<PyObject> {
    // 1. Scambio DH su X25519 (sicurezza classica)
    let ephemeral = EphemeralSecret::random_from_rng(OsRng);
    let eph_pub   = PublicKey::from(&ephemeral);
    let peer_key  = PublicKey::from(
        <[u8;32]>::try_from(x25519_peer_pub).unwrap());
    let dh_shared = ephemeral.diffie_hellman(&peer_key);

    // 2. Incapsulamento Kyber-768 (sicurezza post-quantistica)
    let kyber_pk = kyber768::PublicKey::from_bytes(kyber_peer_pub)?;
    let (kyber_ss, kyber_ct) = kyber768::encapsulate(&kyber_pk);

    // 3. Segreto finale: BLAKE3(X25519_shared || Kyber_shared)
    //    Combinare via hash elimina debolezze strutturali di entrambi
    let mut combined = Vec::new();
    combined.extend_from_slice(dh_shared.as_bytes());
    combined.extend_from_slice(kyber_ss.as_bytes());
    let final_secret = blake3::hash(&combined);
```

```
    let dict = pyo3::types::PyDict::new(py);
    dict.set_item("shared_secret",
        PyBytes::new(py, final_secret.as_bytes()))?;
    dict.set_item("x25519_eph_pub",
        PyBytes::new(py, eph_pub.as_bytes().as_ref()))?;
    dict.set_item("kyber_ciphertext",
        PyBytes::new(py, kyber_ct.as_bytes()))?;
    Ok(dict.into())
}
```

8.2.3 Il Double Ratchet

L'algoritmo **Double Ratchet** garantisce che ogni messaggio venga cifrato con una chiave diversa, derivata deterministicamente dallo stato della sessione. Anche se un attaccante dovesse compromettere la chiave di un singolo messaggio, non potrebbe decifrare i messaggi precedenti (*backward secrecy*) né quelli futuri (*forward secrecy*).

Il ratchet è composto da due livelli:

1. **Symmetric ratchet**: ad ogni messaggio, la chain key avanza di un passo tramite HKDF, generando una message key usa-e-getta.

2. **DH ratchet**: periodicamente, le parti generano nuove coppie DH effimere e rinnovano la root key, garantendo forward secrecy anche in caso di compromissione dello stato.

Listing 8.3: Cifratura con ratchet simmetrico (`lib.rs`).

```
#[pyfunction]
fn ratchet_encrypt(
    py:          Python<'_>,
    state_json:  &str,
    plaintext:   &[u8],
) -> PyResult<PyObject> {
    let mut state: RatchetState = serde_json::from_str(state_json)?;

    // Avanzamento chain key: genera message key usa-e-getta
    let (new_ck, msg_key) = kdf_ck(&state.chain_key_send);
    state.chain_key_send  = new_ck;
    state.send_count      += 1;
```

```
    // Cifratura con ChaCha20-Poly1305 + nonce casuale
    let cipher = ChaCha20Poly1305::new(Key::from_slice(&msg_key));
    let nonce  = ChaCha20Poly1305::generate_nonce(&mut AeadOsRng);
    let mut ct = nonce.to_vec();
    ct.extend_from_slice(
        &cipher.encrypt(&nonce, plaintext)
                .map_err(|e| PyValueError::new_err(format!("{e}")))?
    );
    // ...restituisce ciphertext e nuovo stato serializzato
}
```

8.2.4 Derivazione della chiave master: PBKDF2

La chiave master viene derivata dalla password dell'utente mediante **PBKDF2-SHA512** con **600.000 iterazioni** e un salt casuale a 32 byte. L'elevato numero di iterazioni rende gli attacchi brute-force computazionalmente proibitivi anche per un attaccante dotato di hardware dedicato (GPU, ASIC).

Listing 8.4: Derivazione PBKDF2-SHA512 (`lib.rs`).

```
#[pyfunction]
fn derive_key_pbkdf2(
    py:          Python<'_>,
    password:    &[u8],
    salt:        &[u8],
    iterations:  u32,  // 600_000 in produzione
) -> PyResult<PyObject> {
    use pbkdf2::pbkdf2_hmac;
    let mut key = vec![0u8; 32];
    pbkdf2_hmac::<sha2_crate::Sha512>(password, salt, iterations, &mut
    Ok(PyBytes::new(py, &key).into())
}
```

8.3 Gestione delle chiavi in Python

8.3.1 Il sistema a doppia password

Nebulae implementa un sistema di **doppia password** (*duress password*): esistono due file di identità cifrati su disco, indistinguibili a livello di metadati. Il primo viene sbloccato dalla password reale e contiene i dati effettivi; il secondo viene sbloccato da una password di copertura e contiene dati plausibili ma fittizi. In caso di coercizione, l'utente può rivelare la seconda password senza compromettere i propri dati reali.

Listing 8.5: Sistema a doppia password (`crypto_manager.py`).

```
class DualPasswordManager:
    """
    Due file di identita' cifrati, indistinguibili dall'esterno:
      - identity.a.enc  (sbloccato dalla password reale)
      - identity.b.enc  (sbloccato dalla password di copertura)
    """
    def __init__(self, base_dir: Path):
        self.real_path  = base_dir / "identity.a.enc"
        self.decoy_path = base_dir / "identity.b.enc"

    def unlock(self, password: str) -> Optional[Tuple[dict, bool]]:
        """Restituisce (identity, is_real) o None se errata."""
        identity = IdentityManager(self.real_path).load(password)
        if identity:
            return identity, True   # password reale
        identity = IdentityManager(self.decoy_path).load(password)
        if identity:
            return identity, False  # password di copertura
        return None
```

8.3.2 Azzeramento sicuro della memoria

Una vulnerabilità sottovalutata nella programmazione crittografica è la permanenza dei segreti in memoria oltre il loro ciclo di vita utile. In Python, le stringhe sono immutabili e non azzerabili in modo garantito. **Nebulae** risolve il problema usando bytearray (mutabile) combinato con ctypes.memset per l'azzeramento

deterministico, garantendo che le chiavi non vengano lette da un attaccante con accesso alla memoria (*cold boot attack*).

Listing 8.6: Azzeramento sicuro della memoria (`crypto_manager.py`).

```
def _wipe_bytes(data: bytearray) -> None:
    """Sovrascrive con zeri usando ctypes: il compilatore
    non puo' ottimizzare via questa operazione."""
    if isinstance(data, bytearray) and len(data) > 0:
        ctypes.memset(ctypes.c_char_p(bytes(data)), 0, len(data))
```

Lato Rust, la struct SessionKey implementa il trait ZeroizeOnDrop: la memoria viene azzerata automaticamente quando la struct esce dallo scope, senza richiedere azioni esplicite dal programmatore.

Listing 8.7: Zeroizzazione automatica in Rust (`lib.rs`).

```
#[derive(Zeroize, ZeroizeOnDrop)]
struct SessionKey([u8; 32]);
// Quando questa struct viene droppata, i 32 byte
// vengono automaticamente azzerati dal runtime Rust.
```

8.4 Database cifrato

Tutti i messaggi e i metadati dei contatti vengono archiviati in un database **SQLCipher**: una variante cifrata di SQLite che applica AES-256 a livello di pagina.

8.4.1 Header Blindness: indice HMAC-SHA3

Anche se il contenuto è cifrato, la struttura delle query (chi ha scritto a chi, quando, con quale frequenza) può rivelare informazioni sensibili: è il problema della **metadata leakage**. **Nebulae** lo risolve con l'**Header Blindness**: gli indirizzi onion non vengono mai memorizzati in chiaro nel database, ma trasformati in token opachi mediante HMAC-SHA3-256.

Listing 8.8: Indice HMAC per Header Blindness (`database.py`).

```
class ContactIndex:
    """Converte indirizzi .onion in token HMAC opachi."""
    def __init__(self, master_key: bytes):
        self._key = master_key
```

```
    def index(self, onion_address: str) -> str:
        # L'indirizzo non viene mai memorizzato in chiaro
        token = _rust.hmac_sha3_index(
            self._key, onion_address.encode())
        return token.hex()
```

Listing 8.9: Schema del database cifrato (`database.py`).

```
CREATE TABLE IF NOT EXISTS messages (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    contact_idx    TEXT NOT NULL,        -- token HMAC-SHA3, non l'onion
    direction      TEXT NOT NULL,        -- 'in' | 'out'
    encrypted      BLOB NOT NULL,        -- ChaCha20-Poly1305 ciphertext
    timestamp      REAL NOT NULL,
    uuid           TEXT UNIQUE NOT NULL,
    self_destruct_at REAL DEFAULT NULL -- autodistruzione programmata
);
```

8.4.2 Modalità amnesica

In modalità **amnesic**, il database viene creato interamente in RAM (:memory: SQLite) e non lascia alcuna traccia su disco al termine della sessione.

Listing 8.10: Database amnesico in RAM (`database.py`).

```
class AmnesicDB:
    """Database in-memory: nessuna traccia su disco."""
    def __init__(self):
        self._conn = sqlite3.connect(
            ":memory:", check_same_thread=False)
        self._conn.executescript(SCHEMA)
        self._conn.commit()
```

8.5 Rete P2P su Tor

Il livello di rete è costruito sopra Tor, usando **Hidden Services v3** (indirizzi onion a 56 caratteri, basati su Ed25519) per garantire l'anonimato bidirezionale dei nodi.

8.5.1 Padding adattivo (chaff traffic)

Per contrastare gli attacchi di *traffic analysis* condotti da Eve, **Nebulae** genera continuamente pacchetti fittizi (*chaff*) di dimensione e frequenza casuali. Questo rende molto più difficile dedurre dall'analisi del traffico quando Alice sta comunicando con Bob.

Listing 8.11: Generatore di traffico chaff (`network.py`).

```
class AdaptivePadding:
    """Pacchetti chaff casuali per mascherare i pattern di traffico."""
    def _loop(self) -> None:
        while not self._stop.is_set():
            delay = random.uniform(2.0, 8.0)  # intervallo casuale
            self._stop.wait(timeout=delay)
            size  = random.randint(64, 1024)   # dimensione casuale
            chaff = pack_message(
                MsgType.CHAFF, secrets.token_bytes(size))
            self._send(chaff)
```

8.5.2 Sequenza di handshake

La sequenza di handshake tra due nodi **Nebulae** procede come segue:

1. Il nodo mittente invia ANNOUNCE con le proprie chiavi pubbliche (X25519, Kyber, Ed25519).
2. Il mittente esegue l'incapsulamento ibrido e invia il blob di handshake.
3. Il ricevente decapsula il segreto condiviso e inizializza il proprio stato del Double Ratchet.
4. Da questo momento ogni messaggio viene cifrato con una chiave derivata dal ratchet, unica per quella posizione nella conversazione.

8.6 Panic Wipe: autodistruzione di emergenza

Nebulae implementa una funzione di **panic wipe** che, su comando dell'utente o al termine del timer *canary* (dead-man's switch), distrugge istantaneamente tutti i dati archiviati.

Listing 8.12: Autodistruzione di emergenza (app.py).

```
def panic_wipe(self) -> None:
    """Distrugge tutto e termina il processo."""
    if self.node:
        self.node.panic_wipe()        # azzera chiavi in RAM
    if self.store:
        self.store._db.secure_wipe() # sovrascrittura 3x del DB
    # Sovrascrittura multipla dei file di identita'
    for f in DATA_DIR.glob("identity.*.enc"):
        size = f.stat().st_size
        for _ in range(3):            # 3 passaggi con dati casuali
            f.write_bytes(secrets.token_bytes(size))
        f.unlink()
    sys.exit(0)
```

Capitolo 9

Conclusioni

9.1 Il filo conduttore della crittografia moderna

Come emerge chiaramente dall'analisi condotta in questo documento, la crittografia moderna non è un insieme arbitrario di algoritmi, bensì una **risposta evolutiva a minacce reali e progressive**. Ogni problema aperto ha generato la soluzione che è diventata il fondamento del livello successivo:

Tabella 9.1: Il percorso evolutivo: dal problema alla soluzione.

Problema	**Soluzione introdotta**
Eve può leggere il messaggio	Cifratura simmetrica (AES, ChaCha20)
Come condivido la chiave su canale insicuro?	Diffie-Hellman, scambio asimmetrico
Mallory fa Man-in-the-Middle	Autenticazione + firme digitali (Ed25519)
La chiave viene compromessa in futuro	Forward Secrecy, Double Ratchet
Arriva il computer quantistico	Crittografia ibrida: X25519 + Kyber (ML-KEM-768)

9.2 NEBULAE come sintesi applicata

Il progetto **Nebulae** rappresenta una sintesi concreta di tutti i concetti esaminati in questo documento. Ogni scelta progettuale risponde a una minaccia specifica, come riassunto nella seguente tabella:

Tabella 9.2: Mapping tra concetti teorici e implementazione in **Nebulae**.

Concetto	**Standard**	**Implementazione**
Confidenzialità	ChaCha20-Poly1305	Ogni messaggio e file
Integrità	Poly1305 MAC	Tag AEAD integrato
Autenticità	Ed25519	Firma dell'identità nodo
Scambio di chiavi	X25519	Handshake di sessione
Post-quantum	ML-KEM-768	Handshake ibrido
Forward Secrecy	Double Ratchet	Chiave diversa per messaggio
Key derivation	PBKDF2-SHA512	600.000 it. dalla password
Anonimato rete	Tor Hidden Svc	Indirizzi .onion v3
Privacy metadati	HMAC-SHA3	Indice cieco nel database
Anti-forensics	Zeroize / 3-pass	Panic wipe, modalità amnesica

9.3 Prospettive future

Le settimane successive del percorso di studio approfondiranno i meccanismi qui presentati in modo introduttivo: il funzionamento dettagliato del protocollo TLS 1.3 e della catena di fiducia PKI (Settimana 5), il protocollo X3DH e il Double Ratchet nel dettaglio (Settimana 6), le minacce avanzate e gli attacchi side-channel (Settimana 7), fino alla crittografia post-quantistica e all'algoritmo di Shor (Settimana 8).

Considerazione finale. La sicurezza informatica non è mai un prodotto finito, bensì un processo continuo di adattamento alle minacce emergenti. Il passaggio alla crittografia post-quantistica, che **Nebulae** anticipa già oggi con l'adozione di ML-KEM-768, rappresenta la prossima sfida sistemica per l'intera infrastruttura digitale globale. La standardizzazione NIST del 2024 segna l'inizio di questa transizione.